WORKSPHERE

Enterprise Project Management & Collaboration Platform

Complete Product, Engineering, Architecture & Learning Specification

Document	Version	Purpose
WorkSphere Project Documentation	1.0	Single flagship project for full-stack/job-ready development
Primary Stack	Java + Spring Boot + PostgreSQL + React/Vue	Enterprise web application
Advanced Stack	Redis + Kafka + WebSocket + Docker + CI/CD	Scalability and distributed-systems learning
Audience	Student + Instructor + Development Team	Build, learn, demonstrate, and interview

Executive Summary

WorkSphere is a multi-tenant SaaS platform for organizations to manage teams, projects, tasks, sprints, issues, documents, notifications, collaboration, reporting, subscriptions and audit history. It is intentionally designed as a progressive learning project: the student first builds a clean modular monolith, then adds security, real-time communication, caching, event-driven workflows, observability, containerization, CI/CD and selected microservices.

The project is not intended to teach every technology by forcing every technology into the first version. The architecture evolves in stages. Each stage introduces a business reason for the next concept.

1. Vision and Goals

1.1 Product Vision

Provide a realistic workspace where a company can organize people and software delivery in one place, while giving the student a portfolio project that demonstrates professional engineering practices.

1.2 Learning Goals

- Strong Java fundamentals and object-oriented design.
- Professional Spring Boot backend development.
- Relational database design and PostgreSQL.
- REST API design, validation, pagination, filtering and error handling.
- Authentication, authorization, RBAC, permissions and tenant isolation.
- Modern frontend development with React or Vue and TypeScript.
- Testing at unit, integration and API levels.

- Redis caching and rate limiting.
- Kafka and event-driven architecture.
- WebSocket-based real-time features.
- Docker, Linux, Nginx and CI/CD.
- System design, scalability, observability and production troubleshooting.
- Interview-ready explanation of architecture and engineering decisions.

1.3 Product Goals

- Support multiple organizations from one deployment.
- Prevent cross-tenant data access.
- Provide role- and permission-based access.
- Manage projects, tasks, sprints and issues.
- Provide dashboards and operational reporting.
- Provide real-time and asynchronous notifications.
- Provide auditability for important actions.
- Support subscription plans and payment webhooks as an advanced module.
- Be deployable using containers and automated CI/CD.

2. Scope

2.1 MVP

- Authentication and account management.
- Organization creation and membership.
- Teams and projects.
- Tasks, labels, priorities, assignments and comments.
- Kanban board.
- Sprints and basic reporting.
- Notifications.
- Audit log.
- REST API.
- Frontend dashboard.
- PostgreSQL persistence.
- Automated tests.
- Dockerized local development.

2.2 Advanced Scope

- Multi-tenant isolation hardening.
- Redis caching and rate limiting.
- WebSockets.
- Kafka event bus.
- Background workers.
- File/object storage.
- Email service.

- Subscription and payment integration.
- Advanced analytics.
- Search.
- Observability.
- CI/CD and cloud deployment.
- Selected microservices extraction.

2.3 Explicit Non-Goals for the First Release

- Do not begin with dozens of microservices.
- Do not build a custom payment processor.
- Do not implement a custom identity provider.
- Do not over-engineer database sharding.
- Do not add AI features until core product workflows are stable.
- Do not claim production security simply because JWT and HTTPS exist; security must be tested.

3. Personas and Roles

Role	Primary Responsibilities
Platform Admin	Organizations, platform configuration, plans, support and global audit.
Organization Owner	Organization settings, members, teams, projects, billing.
Project Manager	Projects, sprints, tasks, assignments, reports.
Developer	Assigned work, status updates, comments, files, collaboration.
Employee/Member	Participate in projects and manage assigned work.

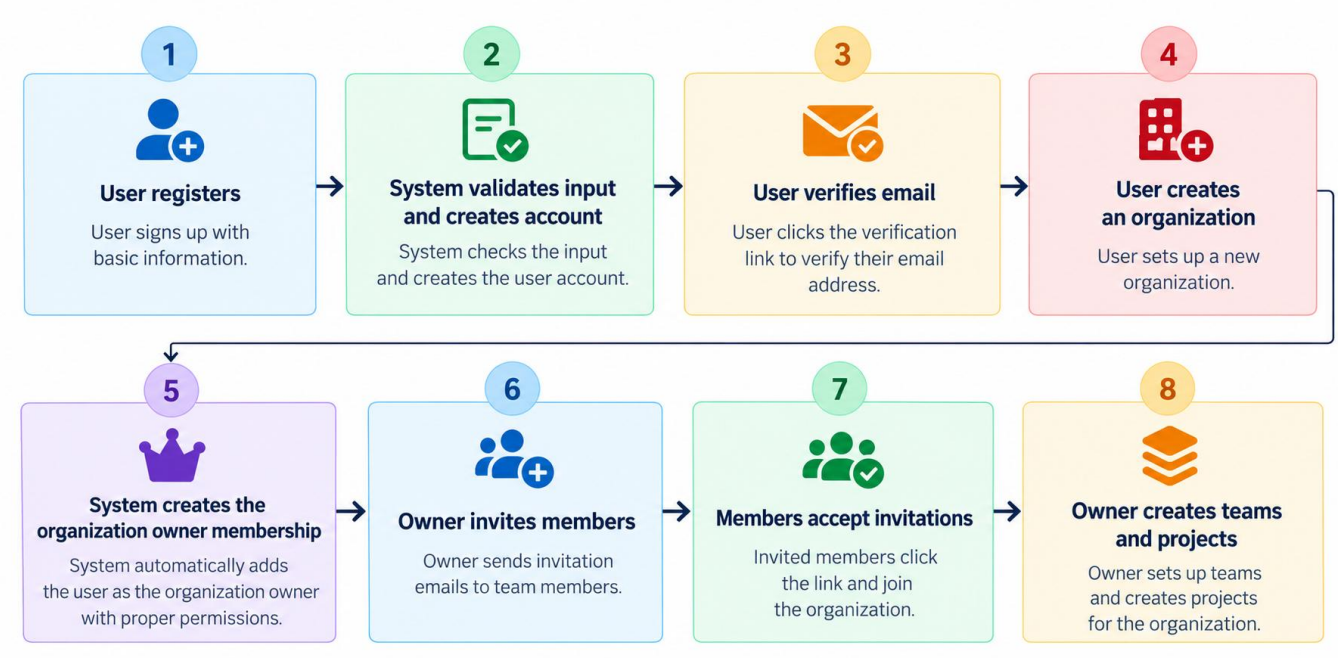
4. Core Functional Modules

Module	Scope
Identity & Access	Registration, login, logout, email verification, password reset, sessions/tokens, roles, permissions.
Organizations	Create organization, membership, invitations, settings, tenant context.
Teams	Create teams, add/remove members, team roles.
Projects	Create, archive, members, settings, project dashboard.
Tasks	CRUD, assignment, priority, labels, due dates, dependencies, attachments, comments.
Kanban	Columns, drag/drop, ordering, status transitions.
Sprints	Backlog, sprint planning, goals, story points, velocity, burndown.
Issues	Bug/issue tracking, severity, priority, environment, lifecycle.
Collaboration	Comments, mentions, real-time updates.
Notifications	In-app, email, real-time and scheduled notifications.
Files	Upload, metadata, access control, object storage.

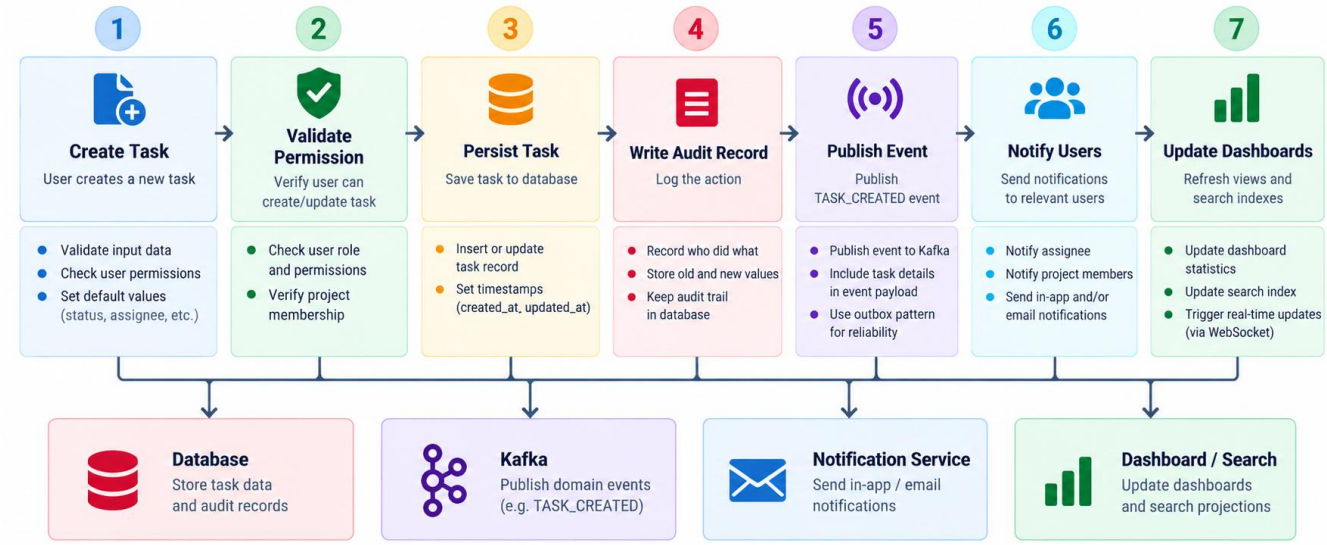
Audit	Immutable-style activity history for security/business actions.
Reporting	Project/team/task metrics and dashboards.
Billing	Plans, limits, subscriptions, payment webhooks.
Search	Global and scoped search.
Administration	Users, permissions, organizations, system configuration.

5. Key User Journeys

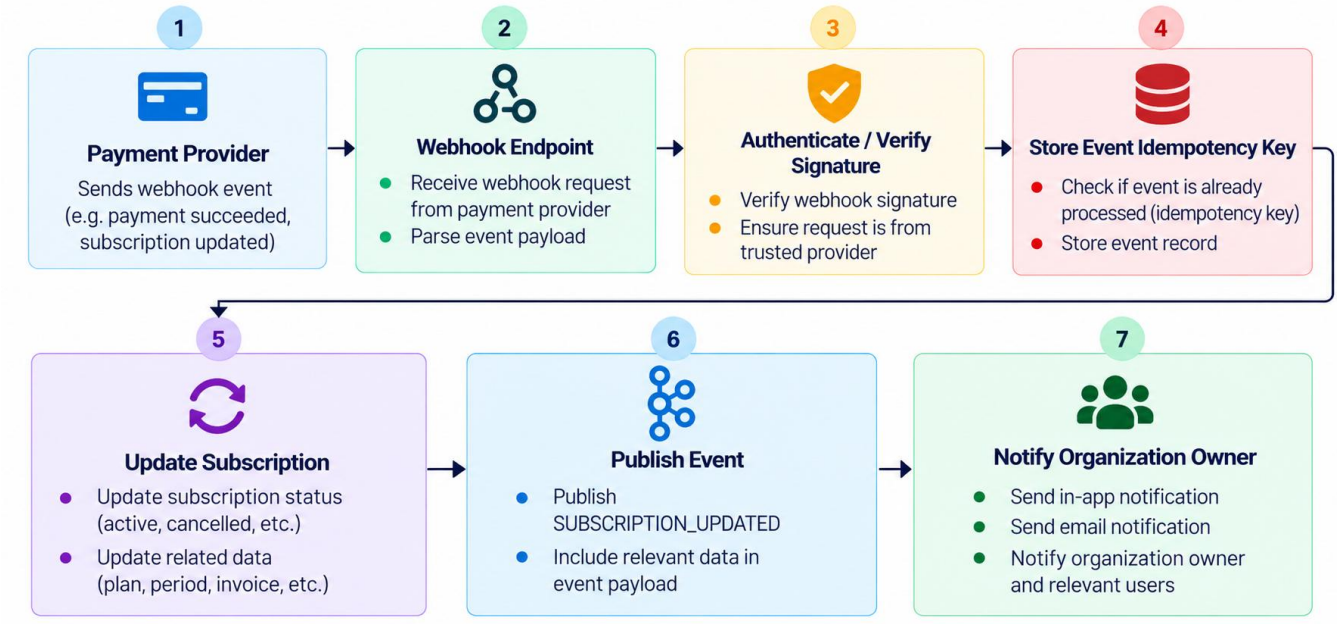
5.1 Organization Onboarding



5.2 Task Lifecycle



5.3 Payment Webhook



6. Functional Requirements

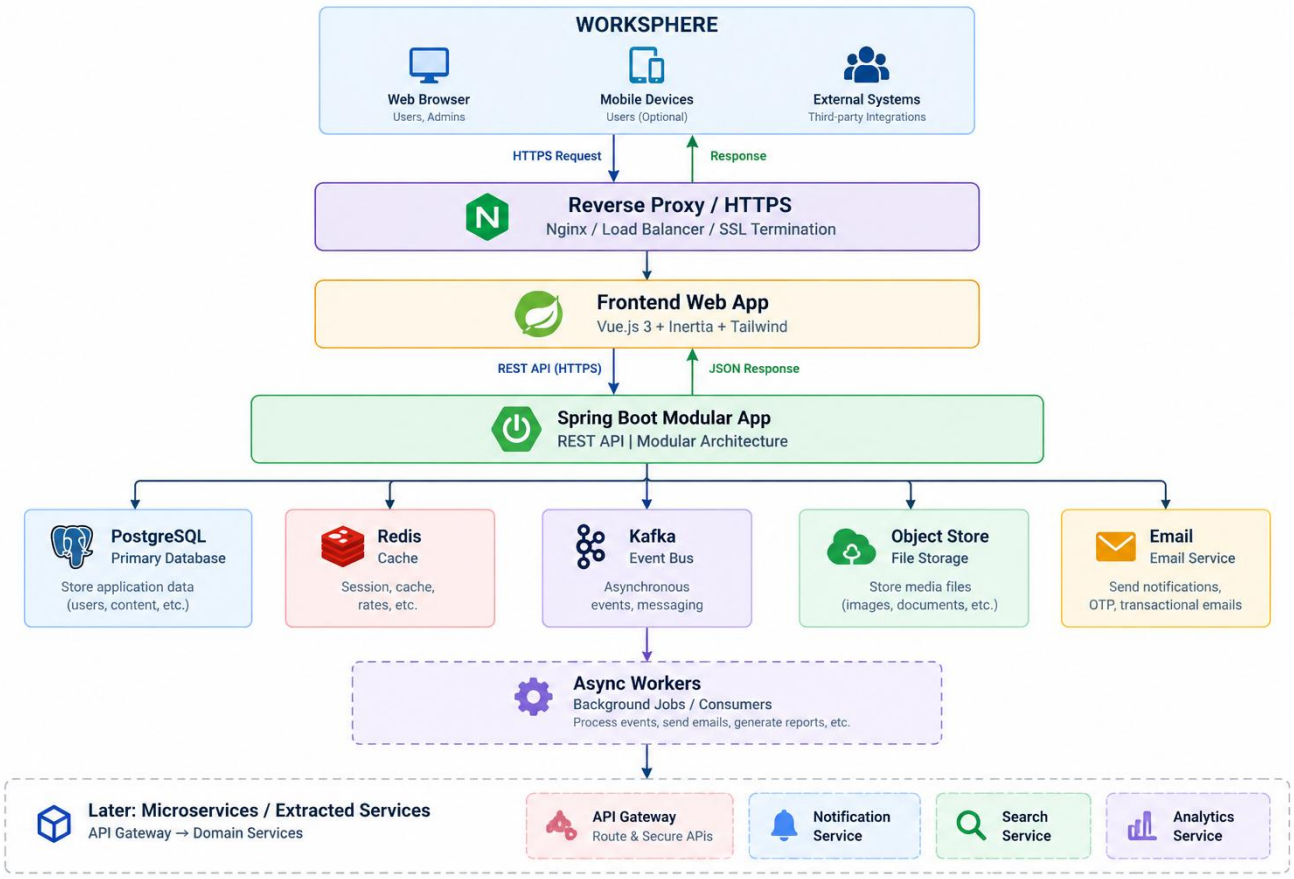
ID	Area	Requirement
FR-001	Authentication	Users shall register, authenticate and recover accounts securely.
FR-002	Authorization	Every protected operation shall

		enforce server-side authorization.
FR-003	Tenant Isolation	Users shall only access organizations and resources to which they are authorized.
FR-004	Projects	Authorized users shall create and manage projects.
FR-005	Tasks	Authorized users shall create, assign, update and track tasks.
FR-006	Sprints	Project managers shall plan and close sprints.
FR-007	Issues	Users shall report and manage issues.
FR-008	Collaboration	Users shall comment and mention other users.
FR-009	Notifications	System shall notify users about important events.
FR-010	Audit	Security/business-sensitive actions shall be auditable.
FR-011	Reporting	System shall provide project and organization metrics.
FR-012	Billing	Advanced version shall support plans and subscription state.

7. Non-Functional Requirements

Area	Target / Principle
Security	Server-side authorization, secure credential handling, HTTPS, validation, auditability.
Availability	Graceful failure and health checks; exact SLA depends on deployment.
Performance	Pagination, indexes, caching and bounded queries; establish measured budgets through testing.
Scalability	Stateless API where practical; horizontal scaling path; async workloads through Kafka/workers.
Maintainability	Modular architecture, tests, code standards and documentation.
Observability	Structured logs, metrics, traces/diagnostics and correlation IDs.
Accessibility	Keyboard navigation, semantic markup, labels, focus management and accessible states.
Data Integrity	Transactions, constraints, optimistic/pessimistic concurrency where appropriate.

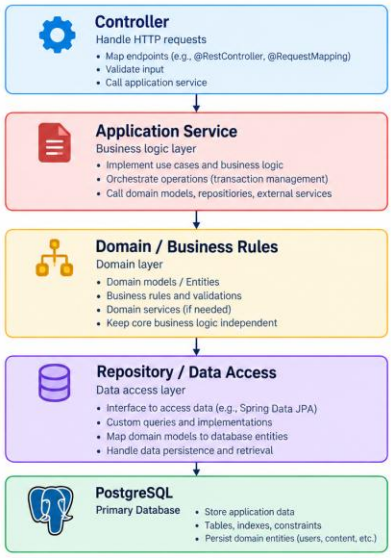
8. High-Level Architecture



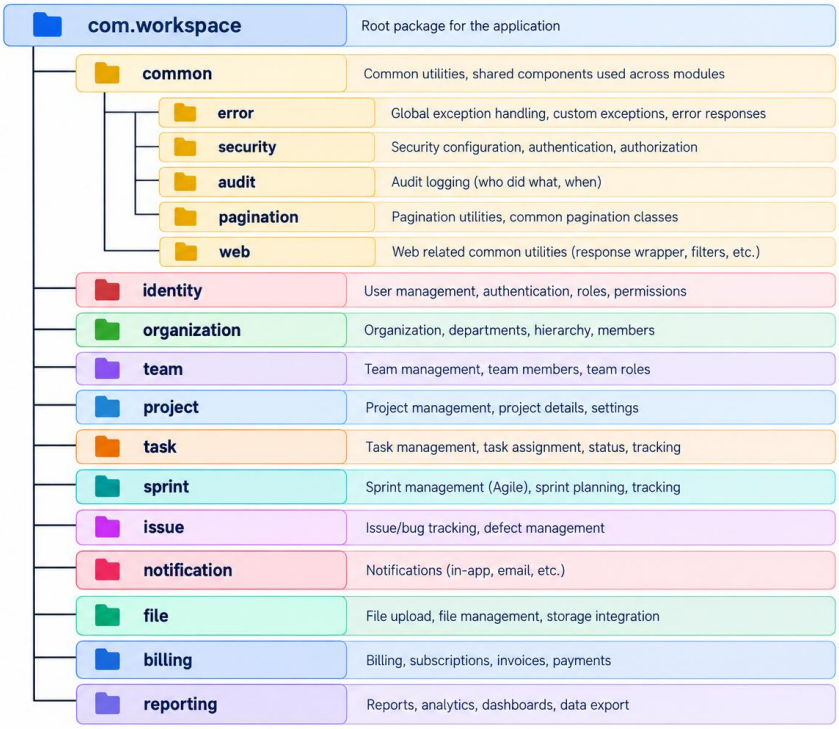
The recommended implementation begins as a modular monolith. Modules are separated by domain boundaries so they can later be extracted without first paying the operational cost of a large microservice estate.

9. Backend Architecture

9.1 Recommended Layers



9.2 Example Package Structure



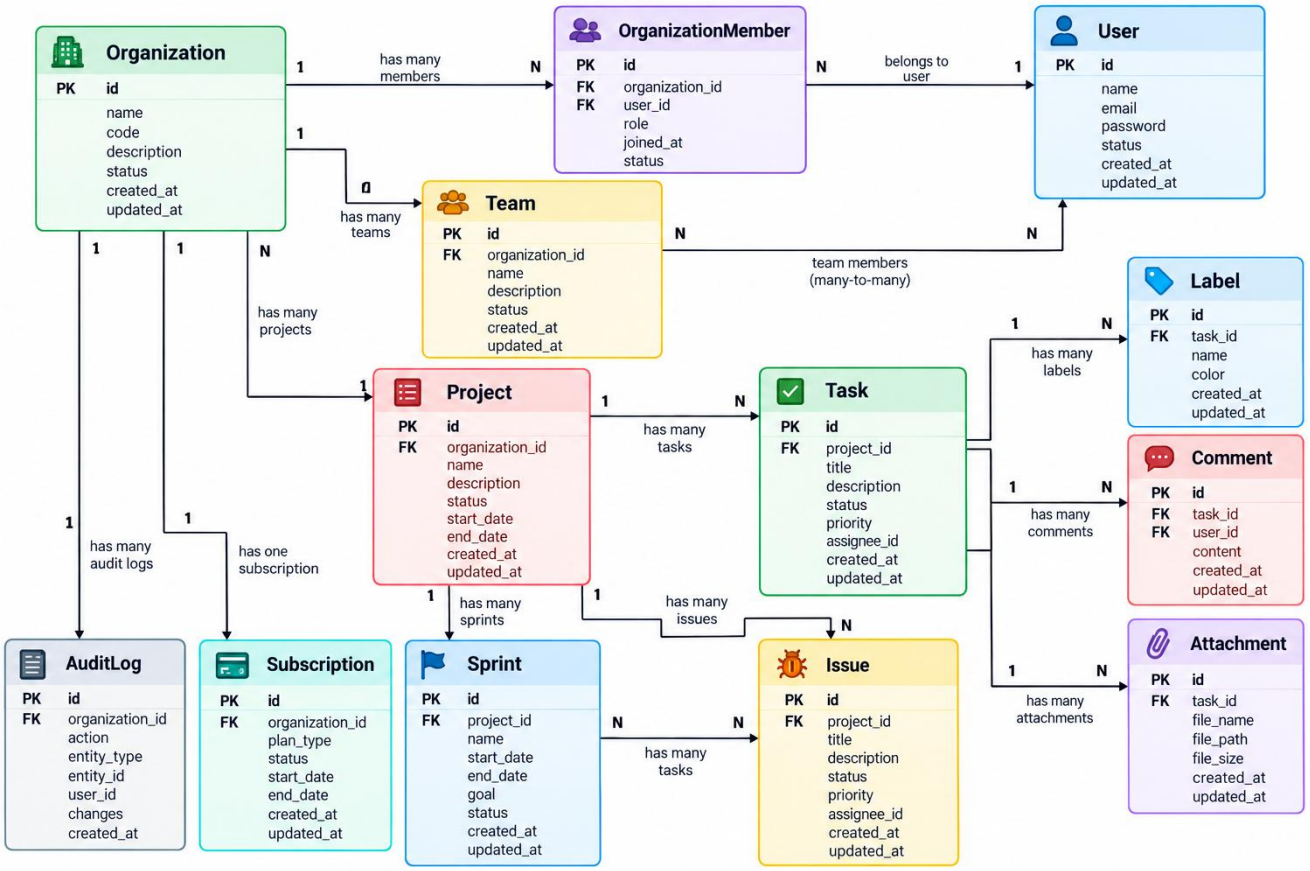
Each module should own its use cases and data-access rules as much as practical. Avoid a giant shared service layer and avoid controllers containing business logic.

10. Database Design

10.1 Core Entities

- users
- organizations
- organization_members
- roles
- permissions
- role_permissions
- teams
- team_members
- projects
- project_members
- tasks
- task_labels
- labels
- task_dependencies
- sprints
- sprint_tasks
- issues
- comments
- attachments
- notifications
- audit_logs
- invitations
- subscriptions
- plans
- payments

10.2 Relationship Model



10.3 Tenant Isolation

The application should include tenant context in authorization decisions and queries. PostgreSQL Row-Level Security can be evaluated as an additional defense-in-depth layer for appropriate tables. PostgreSQL documents that row security policies can restrict which rows a user/session may select or modify, with default-deny behavior when enabled without an applicable policy. ?cite?turn0search4?

Do not rely only on a client-provided organization_id. The server must derive or validate tenant context from the authenticated principal and membership/permission model.

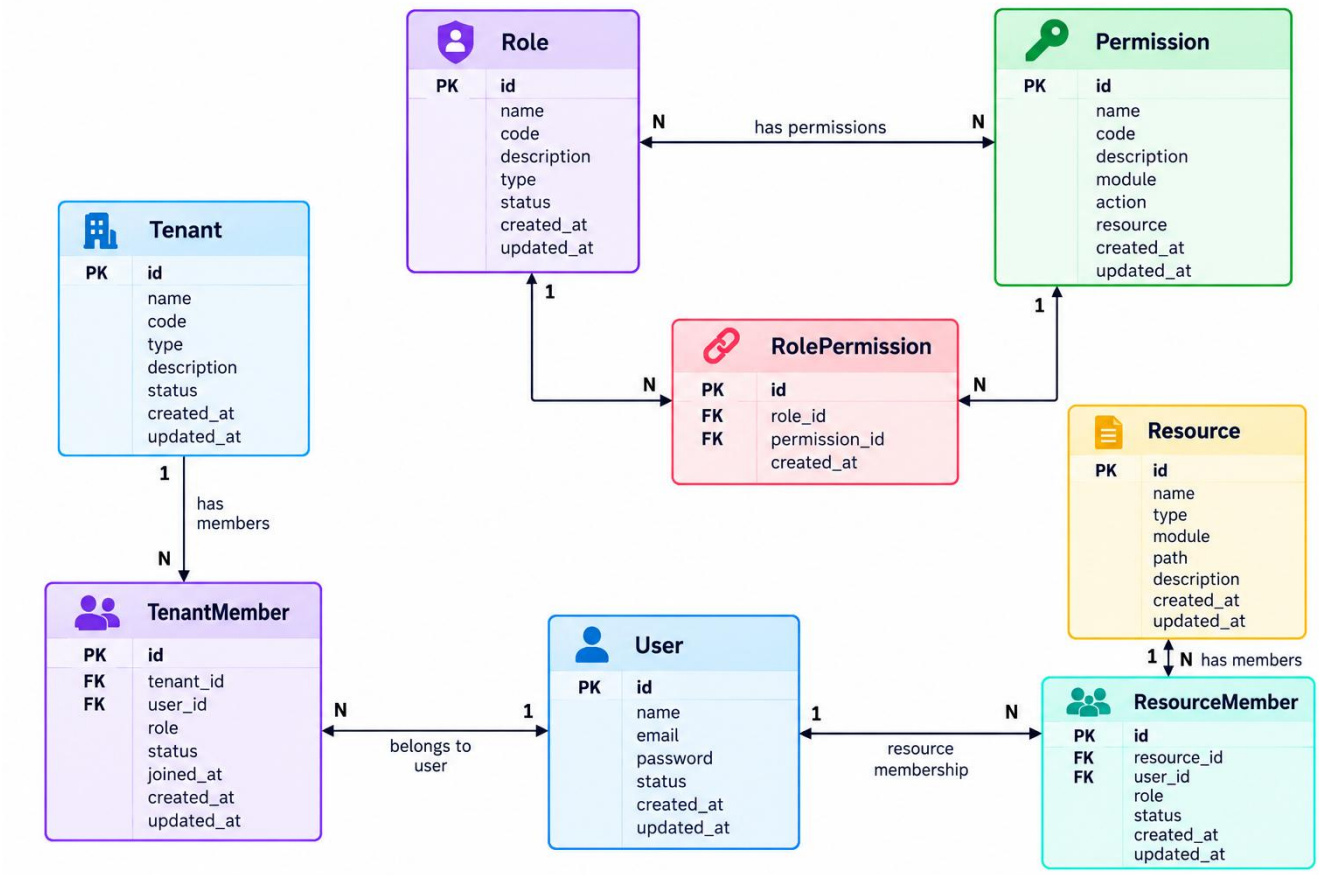
11. Authentication & Authorization

11.1 Authentication

- Registration and email verification.
- Password hashing using an established password-hashing implementation.
- Login with access-token/session strategy.
- Short-lived access tokens where JWT is used.
- Refresh-token rotation/revocation strategy.
- Password reset with expiring, single-use tokens.

- Account lock/risk controls as appropriate.
- Optional MFA as an advanced feature.

11.2 Authorization



Authorization must be enforced on the trusted server side, including resource-level checks. OWASP ASVS explicitly covers function-level, data-specific and field-level authorization, including protections against object-level and property-level authorization failures and cross-tenant access. cite turn0search6 turn0search1

11.3 Security Baseline

- HTTPS in deployed environments.
- Strict input validation.
- Safe output encoding where applicable.
- Parameterized queries/ORM protections.
- Consistent error responses without secret leakage.
- Rate limiting on authentication and sensitive endpoints.
- Audit logging for privileged actions.
- Secure secret management; never commit secrets.
- Content-Type and request-size validation.
- Security headers and safe CORS configuration.
- File upload validation and storage isolation.
- Threat modeling for high-risk flows.

OWASP's current ASVS provides a structured basis for web-application security verification, including authentication, access control, validation, cryptography, logging, data protection, files, APIs and configuration. The project should use ASVS as a checklist rather than treating security as a final-stage activity. cite turn0search3 turn0search8

12. REST API Standards

12.1 URL Convention

```
/api/v1/auth/login
/api/v1/organizations
/api/v1/organizations/{organizationId}/projects
/api/v1/projects/{projectId}/tasks
/api/v1/tasks/{taskId}/comments
```

12.2 API Practices

- Use explicit request and response DTOs.
- Validate input at the API boundary.
- Use consistent HTTP status codes.
- Return stable error codes/messages.
- Support pagination for collections.
- Support explicit filtering and sorting.
- Avoid exposing internal entity structures.
- Document APIs with OpenAPI.
- Use idempotency keys for retry-sensitive operations such as payments.

OWASP REST guidance emphasizes HTTPS, endpoint-level access control and input validation for REST services. These are mandatory design requirements for WorkSphere. cite turn0search12 turn0search1

13. API Inventory

Area	Representative APIs
Auth	POST /auth/register, /auth/login, /auth/refresh, /auth/logout
Users	GET/PUT /users/{id}; profile and preferences
Organizations	POST/GET/PUT /organizations; membership and invitations
Teams	CRUD /organizations/{id}/teams
Projects	CRUD /projects; members; settings
Tasks	CRUD /projects/{id}/tasks; assignment; status; labels
Sprints	CRUD /projects/{id}/sprints; start/complete
Issues	CRUD /projects/{id}/issues
Comments	POST/GET/DELETE /tasks/{id}/comments
Notifications	GET /notifications; mark-read
Reports	GET /projects/{id}/reports/*
Files	POST/GET/DELETE /attachments
Billing	GET /plans; POST /subscriptions; POST

	/webhooks/payment
Audit	GET /organizations/{id}/audit-logs

14. Frontend Architecture

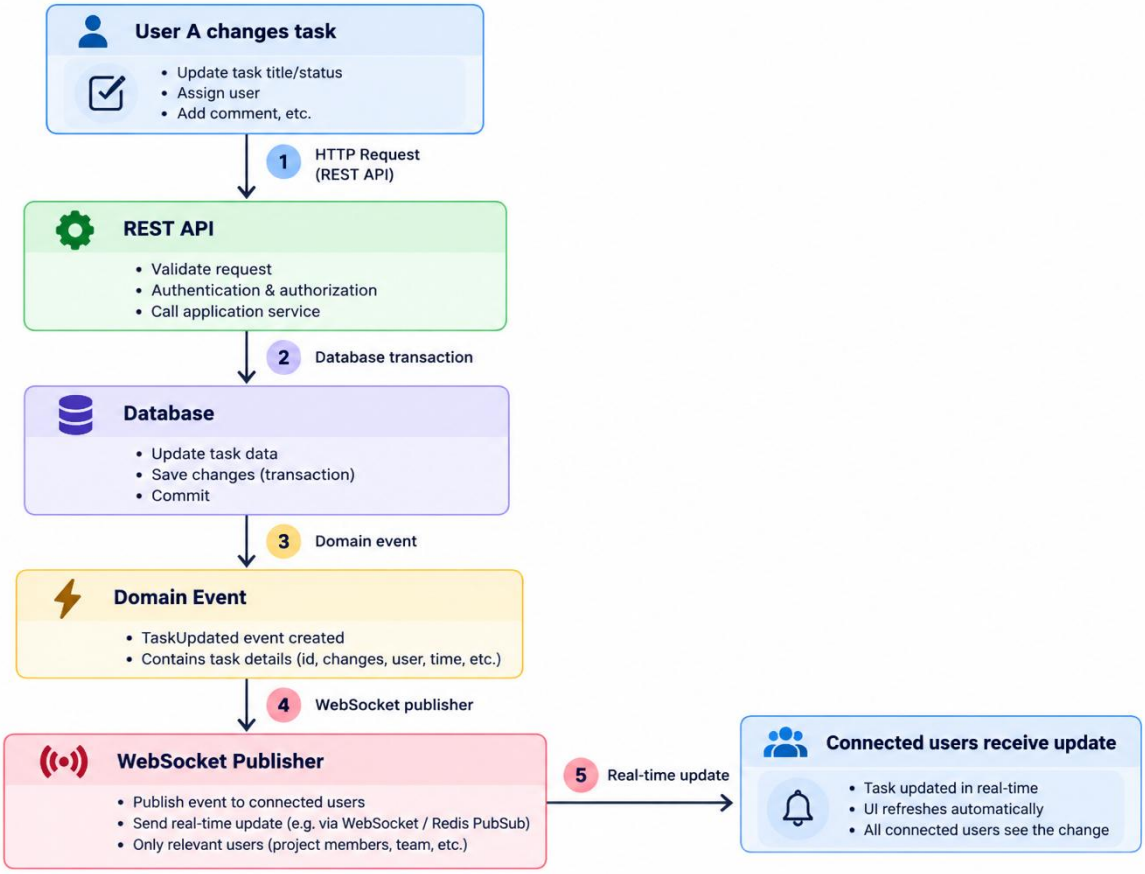
14.1 Screens

- Login / registration / recovery.
- Organization selector.
- Dashboard.
- Projects list.
- Project overview.
- Kanban board.
- Backlog.
- Sprint planning.
- Issue tracker.
- Task details.
- Team directory.
- Notifications center.
- Reports and analytics.
- Settings.
- Billing.
- Administration.

14.2 Frontend Principles

- Reusable components.
- Typed API clients.
- Centralized authentication state.
- Server-state handling separated from local UI state.
- Loading, empty, error and success states.
- Optimistic updates only where consistency behavior is understood.
- Accessible keyboard and focus behavior.
- Responsive design.

15. Real-Time Architecture



Use WebSockets for user-visible low-latency events such as notifications, task-board changes and mentions. Do not use WebSockets for every backend operation; ordinary CRUD remains easier to reason about through REST.

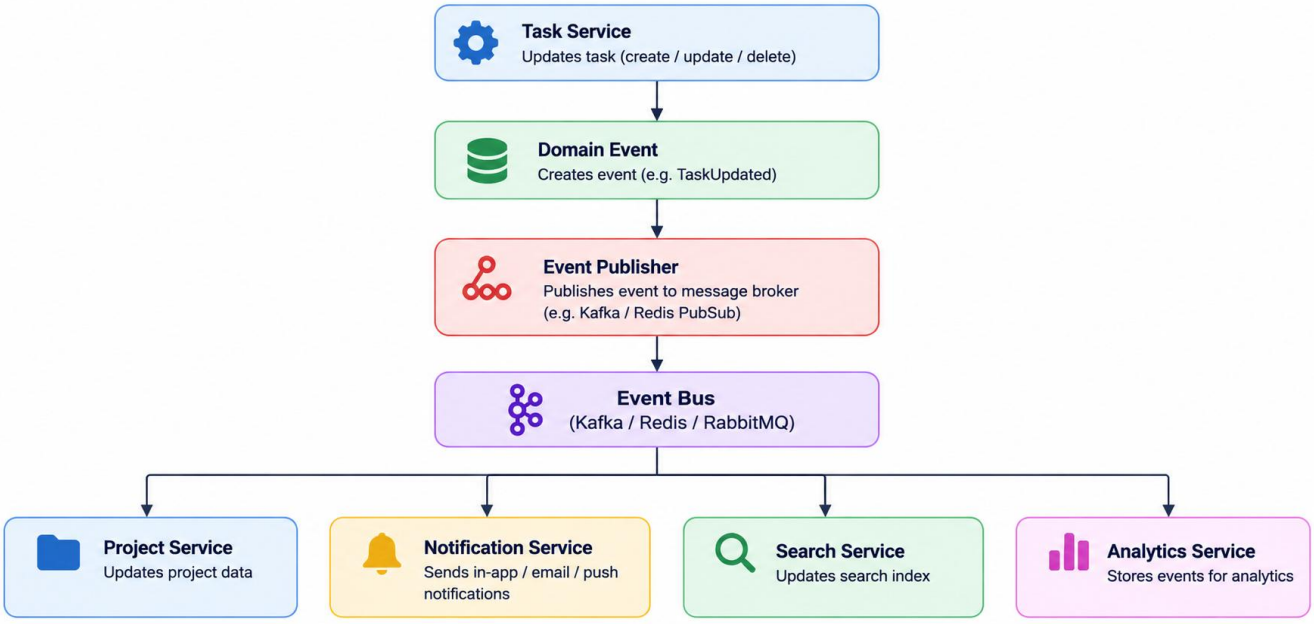
16. Kafka & Event-Driven Architecture

16.1 Events

- USER_REGISTERED
- ORGANIZATION_CREATED
- MEMBER_INVITED
- PROJECT_CREATED
- TASK_CREATED
- TASK_ASSIGNED
- TASK_STATUS_CHANGED
- COMMENT_CREATED
- ISSUE_CREATED
- SPRINT_STARTED

- SPRINT_COMPLETED
- SUBSCRIPTION_UPDATED

16.2 Event Flow



Kafka supports different delivery semantics and consumer-group partitioning; the design must explicitly decide how duplicates, retries and failures are handled rather than assuming that 'Kafka means exactly once'. Kafka's own design documentation distinguishes at-most-once, at-least-once and exactly-once semantics and explains the failure boundaries. For WorkSphere, consumers should be designed to be idempotent, with retries and dead-letter handling where appropriate. cite turn0search10 turn0search13

16.3 Outbox Pattern

For important database-to-event workflows, use a transactional outbox: write the business change and an outbox record in one database transaction, then publish the outbox event asynchronously. This avoids the classic failure where the database commits but event publication fails.

17. Redis Strategy

Use	Example	Invalidation Strategy
Cache	Project summary	Invalidate/update on project/task changes
Rate limiting	Login/API limits	TTL-based
Temporary state	Short-lived workflow data	TTL
Distributed coordination	Carefully selected locks	Short TTL + failure-safe design

18. File Storage

- Store file metadata in PostgreSQL.
- Store binary content in object storage rather than database blobs for normal attachments.
- Generate controlled download/upload URLs where appropriate.
- Validate size, type and content.
- Never trust filename extensions alone.
- Apply authorization before serving private files.
- Record uploads/deletes in audit history.

19. Search

Phase 1: PostgreSQL-based search for projects, tasks and users. Phase 2: introduce a dedicated search engine only if scale or search quality requirements justify it. The student should learn the trade-off instead of adding Elasticsearch simply because it appears in technology lists.

20. Billing & Subscription

- Plans: Free, Professional, Enterprise.
- Plan feature limits.
- Subscription lifecycle.
- Payment creation.
- Webhook verification.
- Idempotent webhook processing.
- Grace periods and cancellation.
- Billing history.



21. Reporting & Analytics

- Open vs completed tasks.
- Overdue tasks.
- Tasks by priority.
- Tasks by assignee.
- Sprint velocity.
- Burndown.
- Cycle/lead-time metrics as an advanced feature.
- Team workload.
- Project health summary.

Reporting queries should be designed separately from transactional workflows when necessary. For higher scale, consider read models or an analytics pipeline rather than repeatedly executing expensive joins on the primary workload.

22. Audit Logging

Field	Purpose
id	Unique audit event
tenant_id	Organization context
actor_id	Who performed the action
action	What happened
resource_type	Affected entity type
resource_id	Affected entity
metadata	Safe contextual details
created_at	When it happened
request_id	Correlation with application logs

23. Concurrency & Data Integrity

- Use database transactions around logically atomic operations.
- Use optimistic locking for entities where concurrent edits are likely.
- Use row-level locking for operations that require serialization, such as carefully designed inventory-like counters.
- Add unique constraints for invariants.
- Make retry-sensitive operations idempotent.
- Never assume frontend ordering equals backend ordering.

PostgreSQL supports row-level locking that blocks conflicting writers/lockers while allowing normal reads, which is useful for carefully selected concurrency-sensitive workflows. cite turn0search14

24. Testing Strategy

Layer	Tools / Approach	Examples

Unit	JUnit + Mockito	Services, validators, business rules
Repository	Spring test support	Queries and persistence behavior
Integration	Testcontainers	API + PostgreSQL + Redis/Kafka
API	REST client / automated suite	Auth, CRUD, authorization
Frontend	Component/unit testing	Forms, board interactions
E2E	Browser automation	Login -> project -> task -> completion
Security	Negative tests	BOLA, privilege escalation, invalid tokens
Performance	Load testing	List/search/dashboard endpoints

24.1 Required Negative Tests

- User from Organization A cannot read Organization B's project.
- Project member without permission cannot delete a project.
- Expired/revoked token cannot access protected data.
- User cannot modify fields they are not authorized to modify.
- Invalid pagination/filter values are rejected safely.
- Oversized/malicious file upload is rejected.
- Duplicate payment webhook does not duplicate subscription state.

25. Observability

- Structured JSON logs in deployed environments.
- Request/correlation ID.
- Health and readiness endpoints.
- Application metrics.
- Database connection metrics.
- Kafka consumer lag monitoring.
- Redis health.
- Error-rate and latency dashboards.
- Alerting for critical failures.

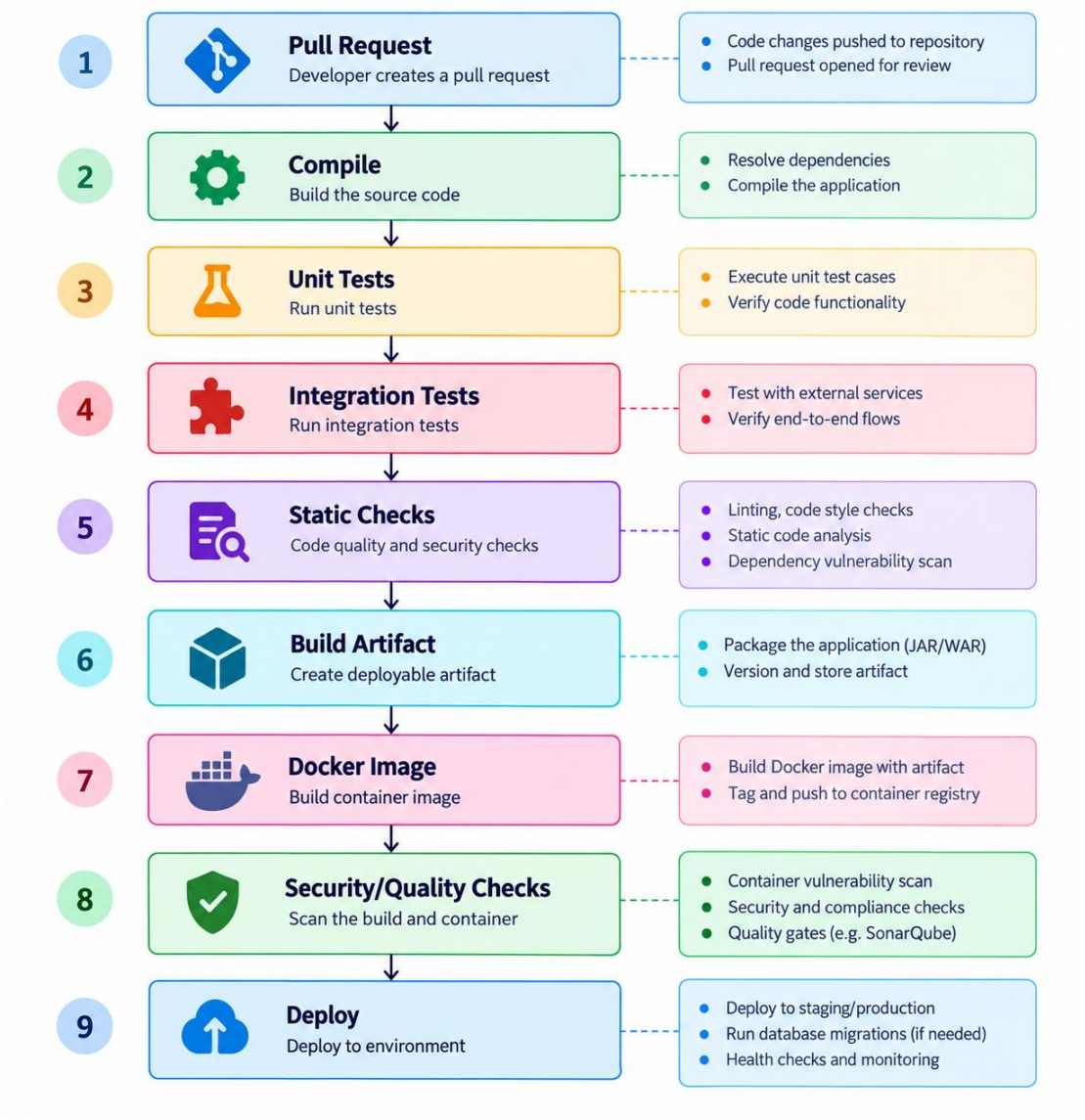
The student should learn to diagnose an incident from logs and metrics rather than simply restarting the server.

26. Docker & Local Development

```
services:
  frontend
  backend
  postgres
  redis
  kafka
  object-storage
  mail-dev
```

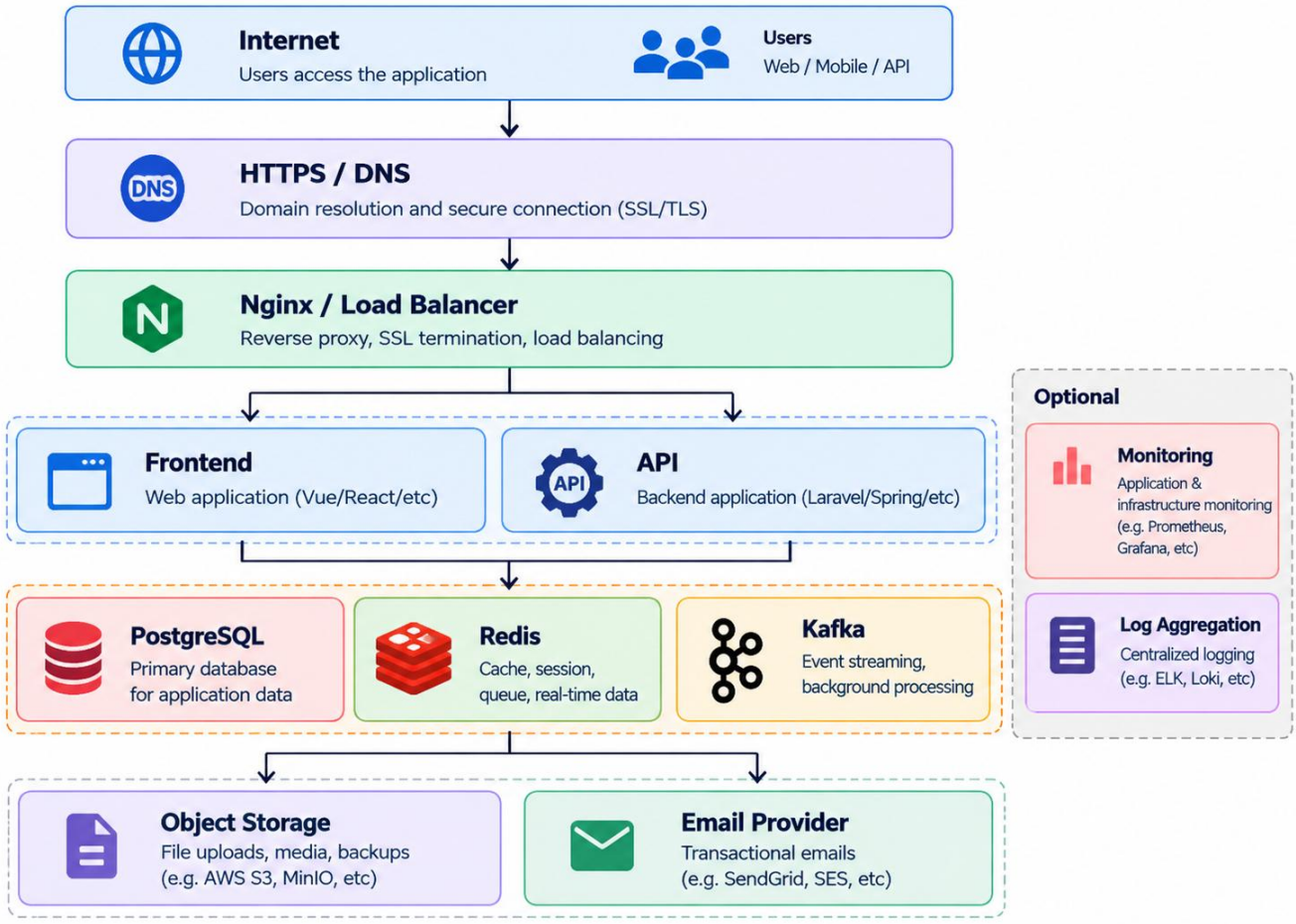
- One command should start the local infrastructure.
- Environment-specific configuration must be externalized.
- Secrets must not be committed.
- Database migrations should run predictably.
- Seed data should be available for demos.

27. CI/CD



- Protected main branch.
- Pull requests required for production changes.
- Automated tests.
- Versioned artifacts.
- Environment-specific deployment.
- Rollback strategy.

28. Deployment Architecture



For learning, the first deployment can be a single cloud VM with Docker Compose. A later version can move to managed PostgreSQL/Redis/Kafka or a container orchestration platform. The objective is to understand the operational trade-offs.

29. Threat Model

Threat	Example	Mitigation
BOLA/IDOR	User changes resource ID in URL	Resource-level authorization and tenant checks
Privilege escalation	Member calls admin endpoint	Server-side permission checks
Cross-tenant leakage	Org A reads Org B	Tenant context + authorization + optional RLS
Credential theft	Token/password exposed	HTTPS, secure storage, short-lived tokens, rotation
Injection	Malicious query/input	Parameterized access + validation
CSRF	State-changing browser request	Appropriate token/cookie/Origin strategy
File abuse	Malicious upload	Validation, limits, isolated storage, authorization

Replay	Duplicate webhook/request	Idempotency keys and event IDs
DoS	Expensive API calls	Rate limits, pagination, query bounds
Secret leakage	API key in Git	Secret manager/environment variables

The security model should be reviewed against OWASP ASVS categories and REST API guidance. ASVS is specifically intended as a basis for testing web-application security controls, and its current structure covers architecture, authentication, authorization, validation, cryptography, logging, data protection, files, APIs and configuration. cite turn0search3 turn0search8

30. Project Development Phases

Phase	Name	Deliverables
Phase 0	Assessment & setup	Assess student, install tools, Git workflow, project standards.
Phase 1	Java foundation	OOP, collections, exceptions, streams, testing basics.
Phase 2	Database	PostgreSQL, SQL, schema, relationships, transactions, indexes.
Phase 3	Spring Boot core	REST, DI, services, repositories, DTOs, validation.
Phase 4	Identity	Registration, login, JWT/session model, roles, permissions.
Phase 5	Organizations	Multi-tenancy, members, teams, invitations.
Phase 6	Projects & tasks	CRUD, business rules, Kanban, comments, labels.
Phase 7	Sprints & issues	Agile workflow, reporting, issue tracking.
Phase 8	Frontend	Dashboard, forms, board, state, API integration.
Phase 9	Testing	Unit, integration, API, negative security tests.
Phase 10	Real-time	WebSocket notifications and board updates.
Phase 11	Redis	Caching, rate limiting, invalidation.
Phase 12	Kafka	Events, consumers, retries, idempotency, outbox.
Phase 13	Files & email	Object storage, asynchronous notifications.
Phase 14	Billing	Plans, subscription state, webhook handling.
Phase 15	Docker & CI/CD	Containers, pipelines, deployment.
Phase 16	Advanced architecture	Extract selected services, gateway, system design.
Phase 17	Interview	DSA, system design, project

		defense, resume/GitHub.

31. Student Learning Map

Skill	Learn	Apply
Java	OOP, collections, generics, streams, exceptions, concurrency	Domain models, services, business rules
SQL	Joins, indexes, transactions, constraints, query design	WorkSphere database
Spring	DI, MVC, REST, validation, configuration	Backend modules
Security	Authentication, authorization, JWT/session, RBAC	Identity module
Frontend	TypeScript, components, forms, routing, state	Dashboard and Kanban
Testing	Unit, integration, API, E2E	Quality gates
Redis	Caching, TTL, rate limiting	Performance and protection
Kafka	Events, consumers, retries, idempotency	Notifications/analytics
WebSocket	Real-time communication	Live notifications/board
Docker	Images, containers, networks, volumes	Local and deployment environment
CI/CD	Build/test/deploy pipeline	GitHub Actions
System Design	Scaling, caching, queues, consistency	Architecture interviews

32. DSA Track — Separate but Parallel

- Arrays and strings.
- Hashing.
- Two pointers.
- Sliding window.
- Linked lists.
- Stack and queue.
- Binary search.
- Recursion and backtracking.
- Heap and priority queue.
- Trees and BST.
- Graphs.
- Greedy algorithms.
- Dynamic programming.

DSA should not be artificially embedded into the product. The student practices DSA separately while using the product to develop engineering judgment and project experience.

33. Definition of Done

- Feature has a clear requirement.
- Database changes have migrations.
- API has validation and authorization.
- Business logic is tested.
- Frontend handles loading/error/empty states.
- Audit/notification behavior is defined where relevant.
- API documentation is updated.
- No secrets are committed.
- Relevant tests pass.
- Code review is completed.
- Feature is deployed to a test environment.

34. Final Portfolio Deliverables

- Live deployed application.
- GitHub repository.
- Architecture diagram.
- Database ER diagram.
- API/OpenAPI documentation.
- README with setup and demo credentials.
- Testing report.
- Security checklist.
- CI/CD pipeline screenshot or link.
- Short demo video.
- Resume-ready project description.
- System-design explanation document.

35. Recommended Build Order



36. Architecture Evolution

Version	Architecture	Why
V1	Modular monolith + PostgreSQL	Learn fundamentals without distributed-system overhead.
V2	Monolith + Redis + WebSocket	Introduce performance and real-time behavior.
V3	Monolith + Kafka + outbox	Introduce reliable asynchronous workflows.
V4	Selected service extraction	Teach service boundaries and operational trade-offs.
V5	Cloud + CI/CD + observability	Demonstrate deployment and production engineering.

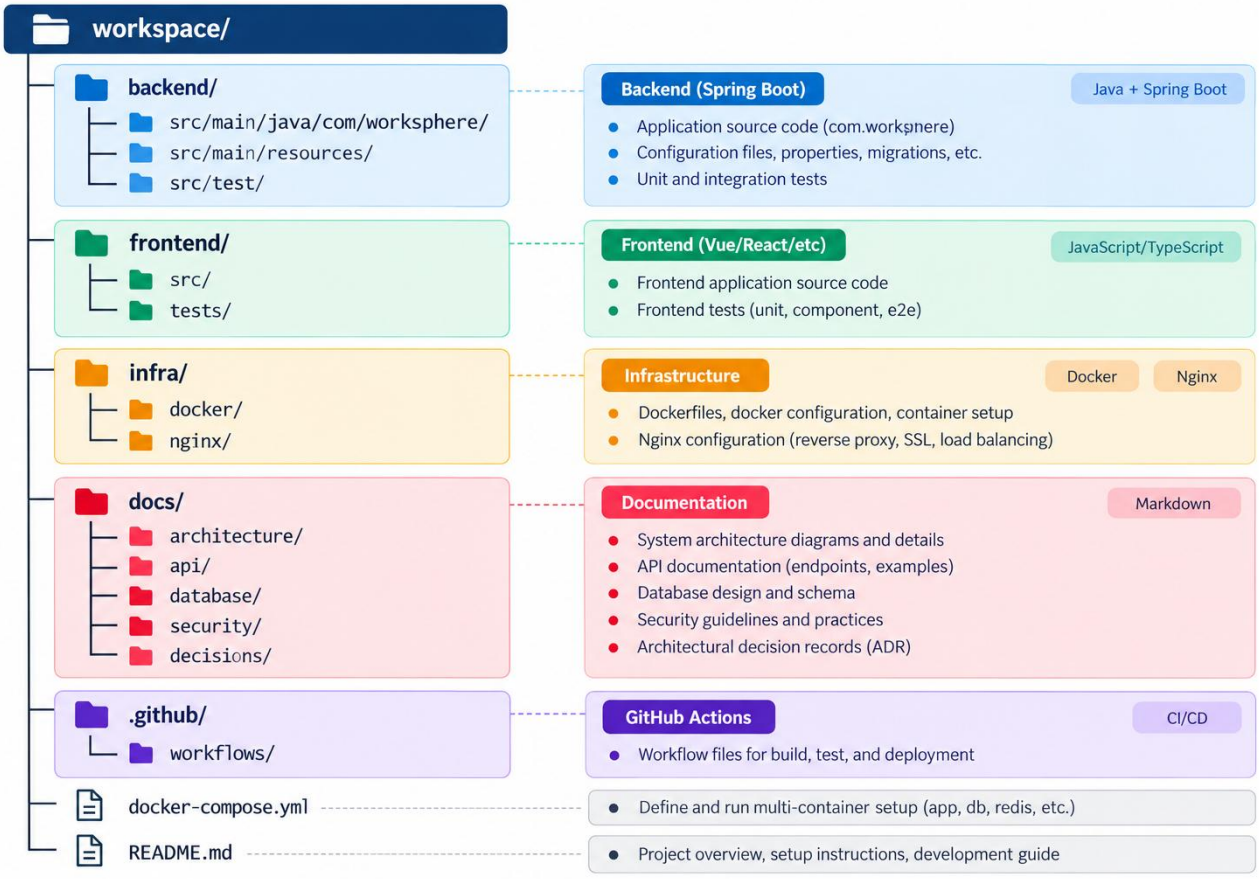
37. Final Success Criteria

The project is considered job-ready when the student can:

- Build a feature from requirement to deployed implementation.

- Design a normalized relational schema.
- Write and optimize SQL queries.
- Build secure REST APIs in Spring Boot.
- Explain authentication and authorization.
- Demonstrate tenant isolation.
- Build a usable TypeScript frontend.
- Write meaningful tests.
- Use Redis appropriately.
- Explain Kafka delivery and failure semantics.
- Design idempotent asynchronous consumers.
- Use Docker and CI/CD.
- Diagnose logs, errors and performance problems.
- Explain architecture trade-offs.
- Solve common medium-level DSA problems under interview conditions.
- Defend every major technology choice in the project.

Appendix A — Suggested Repository Structure



Appendix B — Architecture Decision Records

- ADR-001: Modular monolith first.
- ADR-002: PostgreSQL as primary transactional database.
- ADR-003: Tenant isolation strategy.
- ADR-004: Authentication/token strategy.
- ADR-005: REST API versioning.
- ADR-006: Redis caching policy.
- ADR-007: Kafka event strategy.
- ADR-008: Transactional outbox.
- ADR-009: File storage strategy.
- ADR-010: Deployment strategy.

Each ADR should contain: Context, Decision, Alternatives Considered, Consequences, Security Impact and Operational Impact.

Appendix C — What This Project Proves on a Resume

Resume Signal	Evidence
Backend engineering	Spring Boot modules, REST APIs, transactions, validation
Database skills	PostgreSQL schema, joins, indexes, constraints
Security	Authentication, authorization, tenant isolation, audit
Frontend	TypeScript UI, dashboard, Kanban, real-time state
Distributed systems	Kafka, async workers, idempotency, outbox
Performance	Redis, pagination, query optimization
DevOps	Docker, CI/CD, Linux, Nginx, cloud deployment
Quality	Unit/integration/E2E and security-negative tests
System design	Architecture evolution and scaling decisions
Business understanding	SaaS organizations, subscriptions, billing, workflows

END OF DOCUMENT